Lecture 06: Python Basics (Chapter 4 & 5)

교재 Ch. 4.5, 4.6, 4.7, 5.1

유준수 (Joon Soo Yoo, Ph.D.)

2026년 3월 17일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

4.5 Chained Decisions

여러 갈래 길 중 하나 선택하기 (elif)

왜 if-elif가 필요한가? (if만 쓸 때의 위험성)

단순히 `if`만 여러 개 나열하면, 의도치 않게 **여러 개의 길이 동시에 열릴 수 있음**

잘못된 성적 보정 로직 (score이 60일 때)

코드 (Multiple if)	실행 결과 (Trace)
`score = 60`	초기값: 60
`if score < 70: score += 10`	60 < 70이므로 **70**이 됨
`if 70 <= score < 85: score += 5`	**방금 70이 되었으므로 이 조건도 참!** → **75**

결과 분석

보너스 점수를 한 번만 줘야 하는데, 두 번 다 적용되어 버렸음. 이런 상황을 방지하려면 **배타적 선택(Exclusive Choice)**인 `elif`를 써야 함

elif: "그게 아니라면 혹시 이거니?"

elif는 **else if**의 줄임말입니다. 이전 조건이 False일 때만 다음 조건을 확인합니다.

if-elif-else 구조

```
if condition_1:
    # 조건 1이 True일 때 실행
elif condition_2:
    # 조건 1은 False이고, 조건 2가 True일 때 실행
else:
    # 앞의 모든 조건이 False일 때 실행 (나머지)
```

- 위 사슬(Chain)에서 **오직 한 개의 블록만** 실행됩니다.

if vs elif: 독립적 분기 vs 연결된 사슬

Multiple `if` (독립적)

```
if condition_1: # Test!
  Body 1
if condition_2: # Test!
  Body 2
if condition_3: # Test!
  Body 3
```

- **모든 조건을 각각 검사**
- 위가 True여도 아래를 또 체크
- **여러 블록이 동시 실행 가능**

Chained `elif` (사슬형)

```
if condition_1: # Test!
  Body 1
elif condition_2: # Skip?
  Body 2
elif condition_3: # Skip?
  Body 3
```

- **사슬처럼 연결된 구조**
- 하나만 True면 나머지는 무시
- **오직 한 개의 블록만 실행**

효율성(Performance)

`if` 나열은 컴퓨터가 모든 문을 다 두드려봐야 하지만, `elif` 사슬은 답을 찾는 순간 나머지는 건너뜀. 이것이 바로 **'효율적인 알고리즘'**의 시작임.

실전 예시: 시간에 따른 인사말

시간대 판별 로직

```
if hour < 12:
    print("Good morning")
elif hour < 13:
    print("Lunchtime")
elif hour < 17:
    print("Good afternoon")
else:
    print("Good evening")
```

팁: 순서가 생명!

`elif` 체인은 위에서부터 아래로 검사함. 가장 좁은(구체적인) 범위를 위쪽에 두는 것이 좋은 설계임.

4.6 Nested Decisions

조건문 안의 조건문 (계층적 구조)

중첩(Nesting)이 왜 필요한가?

여러 조건을 동시에 체크해야 할 때, 코드가 중복되는 것을 방지하고 논리를 계층적으로 조직할 수 있습니다.

중복된 코드 (나열형)

```
if game == 1 and p < 2:
  print("Lack")
if game == 1 and p > 4:
  print("Full")
# "game == 1"을 계속 반복
```

개선된 코드 (중첩형)

```
if game == 1:
   if p < 2:
      print("Lack")
   elif p > 4:
      print("Full")
```

- **들여쓰기(Indentation):** 안쪽의 `if`문은 바깥쪽 `if`문이 `True`일 때만 검토됩니다.

중첩 조건문의 실행 흐름

중첩 구조 분석

계층	역할
Outer If	큰 범주의 조건을 먼저 필터링 (예: 게임 종류 선택)
Inner If	세부적인 조건을 판별 (예: 인원수 확인)

보안 팁

보안 시스템에서는 먼저 **ID가 존재하는지** 확인(Outer)하고, 그 다음 **비밀번호가 맞는지** 확인(Inner)하는 식으로 중첩 구조를 설계하여 효율성을 높임.

4.7 Conditional Expressions

한 줄로 끝내는 조건문 (Ternary Operator)

조건부 표현식이란?

`if-else` 구조를 아주 간결하게 표현한 것으로, 보통 변수에 값을 즉시 할당할 때 사용합니다.

기본 템플릿 (Template)

참일 때 값 `if` **조건** `else` **거짓일 때 값**

- **작동 순서:** 먼저 중앙의 **조건**을 검사합니다.
- `True`이면 왼쪽의 값을 선택하고, `False`이면 오른쪽의 값을 선택합니다.

비교: 두 숫자 중 큰 값 찾기 (Max)

[일반 if-else] `if x < y: max_v = y; else: max_v = x`

[조건부 표현식] `max_v = y if x < y else x`

실생활 예시: 더 친숙해지기

단순한 로직일수록 조건부 표현식의 위력이 발휘됩니다.

조건부 표현식 활용 사례

상황	코드 표현
홀짝 판별	`res = "even" if x % 2 == 0 else "odd"`
성적 합격 여부	`status = "Pass" if score >= 60 else "Fail"`
보안 레벨 설정	`access = "Admin" if is_root else "User"`

주의사항

가장 낮은 연산 우선순위를 가집니다. 계산식과 섞어 쓸 때는 괄호를 적절히 사용하여 의도를 명확히 하세요.

우선순위의 함정 예시 (Operator Precedence)

조건부 표현식은 파이썬의 거의 모든 연산 중 **가장 낮은 우선순위**를 가집니다. 즉, 주변의 산술 연산과 비교 연산이 **모두 끝난 뒤**에야 비로소 선택이 일어납니다.

파이썬 주요 연산 우선순위

순위	연산자 종류	비고
1	`( )`, `[ ]`	괄호 (절대 권력)
2	`**`, `*`, `/`, `+`, `-`	산술 연산 (계산 먼저!)
3	`==`, `!=`, `<`, `>`	비교 연산 (참/거짓 판별)
최하위	`A if condition else B`	**조건부 표현식 (최종 선택)**

우선순위의 함정

예시: 계산과 섞였을 때 (Case Study)

`total = fee + 10 if hours > 12 else 2`

1. **컴퓨터의 해석:** `(fee + 10) if hours > 12 else 2`
2. **주의:** `fee + (10 if hours > 12 else 2)` 가 **아닙니다!**

팁: "괄호는 보험이다"

연산 순서가 헷갈릴 때는 무조건 **괄호**()를 쓰세요. `res = x + (10 if is_ready else 5)` 처럼 쓰면 의도가 명확해지고 버그도 예방됩니다.

Chapter 5. Loops

5.1 While Loop (조건부 반복)

반복문(Loops)이란 무엇인가?

동일하거나 유사한 작업을 여러 번 수행해야 할 때 사용하는 코드 블록입니다.

- **데이터의 예:** 학급 명부에서 학생 이름을 한 명씩 꺼내어 출석부 출력

루프의 두 종류

- **while loop:** 조건이 `True`인 동안 계속 반복
- **for loop:** 정해진 횟수나 데이터 범위만큼 반복

while 문의 구조와 흐름

조건식(Loop Expression)이 **True**인 동안 몸체(Loop Body)를 실행합니다.

while 문 기본 문법

```
while condition:
    # 반복할 코드 (Body)
    # 조건에 변화를 주는 코드 (Update)
```

실행 순서

단계	동작
1	조건을 검사한다. (`True`면 단계 2로, `False`면 종료)
2	몸체(Body) 코드를 실행한다.
3	다시 1단계(조건 검사)로 돌아간다.

카운터를 이용한 반복 (Counting)

변수의 값을 하나씩 증가시키거나 감소시키며 원하는 횟수만큼 반복합니다.

1부터 10 사이의 홀수 출력하기

```
counter = 1   # 초기화
while counter <= 10:   # 조건
    if counter % 2 == 1:
        print(counter)
    counter += 1   # 업데이트 (Step size)
```

무한 루프(Infinite Loop) 주의!

만약 `counter += 1`을 빼먹는다면? 조건이 영원히 `True`가 되어 프로그램이 종료되지 않습니다!

예시: 암호 입력 대기 (Sentinel Value)

Mission: "begin"이 입력될 때까지 계속 입력받는 프로그램을 만드세요.

1. `while` 루프를 사용하여 사용자 입력을 반복해서 받습니다.
2. 사용자가 `"begin"`이라고 입력하면 루프를 종료합니다.
3. 종료 시 "The while loop condition has been met."을 출력하세요.

실습 힌트

```
user_input = ""
while user_input != "begin":
    user_input = input("Enter word: ")
```

Any Questions?